

AIOps Sentinel: Self-Healing Cloud

Comprehensive Project Documentation & Technical Blueprint

Engineering Capstone Project

September 2026

Contents

1	1. Executive Summary & Project Vision	3
1.1	1.1 Overview	3
1.2	1.2 The Concept of “Self-Healing Cloud”	3
1.3	1.3 The Ultimate Goal	3
2	2. Problem Statement & Target Audience	4
2.1	2.1 Why Modern Cloud Systems Fail	4
2.2	2.2 Target Audience & User Personas	4
3	3. Why Existing Solutions Are Insufficient	5
4	4. Powerful Innovation Keywords (Core Features)	6
4.1	4.1 AI Incident Clustering — Alert Storm Reduction	6
4.2	4.2 Root-Cause Graph Engine — GNN-Based Causal Inference	6
4.3	4.3 Safe, Closed-Loop Remediation — Policy-Gated Execution	6
4.4	4.4 LLM-Powered “Situation Room” — Natural Language Explanations	6
4.5	4.5 Predictive Failure Scoring — Proactive Maintenance	6
5	5. End-to-End Example Scenario (The User Journey)	7
6	6. System Design & Reference Architecture	8
6.1	6.1 Architecture Pipeline Overview	8
6.2	6.2 Layer-by-Layer Breakdown	8
6.3	6.3 Inter-Layer Interaction Summary	9
7	7. Deep Research: Datasets, Metrics & Research Gap	10
7.1	7.1 Candidate Datasets	10
7.2	7.2 Target Evaluation Metrics	10
7.3	7.3 Research Gap & IEEE Publication Angle	10
8	8. AI/ML Core Functional Modules	12
9	9. Tech Stack & Database Selection	13
9.1	9.1 Recommended Technology Stack	13
9.2	9.2 Database Selection: Supabase vs. Firebase	13
9.3	9.3 Database Schema (SQL)	14
10	10. UI/UX Specifications (For Stitch AI / Lovable)	16
10.1	10.1 Design System	16
10.2	10.2 Pages & Navigation (7 Pages)	16

10.3	10.3 Page-Level Interactions	16
11	11. Risks, Constraints & Mitigation	18
12	12. Conclusion & PPO Pitch Strategy	19
12.1	12.1 Why This Project Is Industry-Ready	19
12.2	12.2 Pitching This to Recruiters for a PPO	19

1. Executive Summary & Project Vision

1.1 Overview

AIOps Sentinel is a capstone engineering platform that reimagines how modern organizations detect, understand, and resolve cloud infrastructure failures. Instead of treating monitoring, alerting, diagnosis, and remediation as four disconnected activities scattered across four different tools, AIOps Sentinel unifies them into a single **closed-loop intelligence system** that watches a distributed system’s telemetry, reasons about *why* something is failing, explains that reasoning in plain English, and — within tightly governed safety boundaries — takes corrective action automatically.

The platform is designed as a **B2B observability intelligence layer**: it does not replace existing tools such as Prometheus, Grafana, or Datadog; it sits *above* them, consuming their telemetry streams and converting raw signals (metrics, logs, traces) into a single, high-confidence, human-readable narrative of “what is going wrong, why it is going wrong, and what should be done about it.”

1.2 The Concept of “Self-Healing Cloud”

A **Self-Healing Cloud** is an infrastructure paradigm in which the system possesses three escalating capabilities:

1. **Self-Awareness** — the ability to continuously model its own “normal” behavior across services, so that deviations are detected as they emerge rather than after they cause visible outages.
2. **Self-Diagnosis** — the ability to trace an observed symptom (e.g., elevated latency) backward through a dependency graph to its true root cause (e.g., a memory leak in an upstream service), rather than surfacing only the symptom.
3. **Self-Repair** — the ability to execute a pre-approved, policy-gated corrective action (restart, rollback, scale-out, circuit-break) without waiting for a human to be paged, read logs, and manually type a command — while still keeping a human in the loop for anything above a defined risk threshold.

AIOps Sentinel implements all three layers, with the third layer deliberately constrained by a **Safety Guardrail** (built on Open Policy Agent) so that automation never outpaces trust.

1.3 The Ultimate Goal

The project’s north star is a measurable shift in incident response posture:

From (Reactive)	To (Proactive, AI-Driven)
Engineers are paged after customers notice the outage	The platform predicts failure risk before it becomes an outage
Hundreds of disconnected alerts flood a Slack channel	Alerts are clustered into one coherent “situation”
Root cause is found by manually reading dashboards for 30–60 minutes	Root cause is surfaced automatically via causal graph inference in seconds
Remediation requires a human to log in and run commands	Remediation is proposed, explained, and executed with one click, inside guardrails
Postmortems are written manually days later	The system logs a structured incident timeline that seeds the postmortem automatically

This shift is quantified later in Section 7 through explicit target metrics such as **Mean Time to Resolution (MTTR) reduction of 40–50%**.

2. Problem Statement & Target Audience

2.1 Why Modern Cloud Systems Fail

Modern applications are no longer monoliths — they are **distributed meshes of dozens to thousands of microservices**, deployed on Kubernetes, communicating over ephemeral networks, and scaled elastically. This architecture, while flexible, introduces failure modes that are structurally different from — and harder to resolve than — traditional single-server failures:

1. **Kubernetes Complexity.** A single user-facing failure can stem from a pod eviction, a misconfigured **resource limit**, a failing liveness probe, a DNS resolution issue inside the service mesh, or a node-level disk pressure event. No single dashboard shows all of these dimensions together.
2. **Alert Storms.** A single root cause (e.g., a database connection pool exhaustion) can trigger **hundreds to thousands of downstream alerts** across every dependent microservice, because monitoring tools are typically threshold-based and per-service, with no concept of causal relationships between alerts.
3. **Hidden Dependencies.** Service ownership is decentralized; the team that owns **checkout-service** frequently does not know that it silently depends on a shared caching layer owned by another team. When that shared dependency degrades, the blast radius is invisible until multiple teams are independently paged.
4. **Ephemeral Infrastructure.** Containers are created and destroyed in seconds. By the time an engineer opens a terminal to inspect a failing pod, the pod may already have been rescheduled, destroying the forensic evidence.
5. **Human Response Is Too Slow.** Industry data consistently shows that the dominant contributor to MTTR is not fixing the problem — it is **finding** the problem. Human cognitive bandwidth cannot parse thousands of time-series signals during a live incident while under the stress of a production outage.

2.2 Target Audience & User Personas

Persona	Role Summary	Core Pain Point Addressed
Site Reliability Engineer (SRE)	On-call engineer responsible for uptime and incident response	Drowning in alert noise; needs one situation, not 500 tickets
DevOps Engineer	Owns CI/CD pipelines and deployment health	Needs to know instantly if a new deployment caused a regression
Platform Team	Maintains the shared infrastructure (Kubernetes clusters, service mesh, databases)	Needs cross-team visibility into hidden dependency failures
Incident Response / NOC Team	First responders coordinating major incidents	Needs a single “Situation Room” view instead of ten open dashboards
Engineering Managers / VP Eng	Accountable for reliability SLAs and postmortems	Needs auto-generated timelines and metrics for retrospectives

3 3. Why Existing Solutions Are Insufficient

Current observability and incident-management tooling addresses **fragments** of the problem, not the whole lifecycle. The table below breaks down each category of existing tool and its specific limitation:

Tool Category	Examples	What It Does Well	Where It Falls Short
Metrics/Logs/Traces platforms	Prometheus, Grafana, Datadog, New Relic	Collect and visualize raw telemetry	Shows symptoms , not causality — an engineer must manually correlate 10 dashboards to guess the root cause
Alerting systems	PagerDuty, Opsgenie	Route alerts to on-call engineers	No deduplication by <i>cause</i> ; a single root issue generates dozens of independent pages
Runbook repositories	Confluence, Notion, internal wikis	Document tribal knowledge	Static text; requires a stressed human to find, read, and correctly execute the right runbook manually
Ticketing/ITSM systems	Jira, ServiceNow	Track incident lifecycle for compliance	Separates diagnosis (in dashboards) from remediation (in scripts/terminals) from documentation (in the ticket) — three disconnected systems
Basic automation scripts	Cron jobs, simple auto-scalers	Handle known, narrow failure patterns (e.g., CPU-based autoscaling)	No causal reasoning; cannot generalize to novel, multi-service failure patterns; no safety policy layer

The core gap: none of these tools close the loop end-to-end — from *raw signal* → *clustered incident* → *causal explanation* → *safe automated action* → *learned postmortem*. AIOps Sentinel is architected specifically to close that loop.

4 4. Powerful Innovation Keywords (Core Features)

4.1 4.1 AI Incident Clustering — Alert Storm Reduction

Using unsupervised clustering (temporal proximity + service-topology proximity + textual similarity of alert payloads via embeddings), the platform groups hundreds of raw alerts that share a common origin into a **single “Situation”**. A 500-alert storm is compressed into 1 actionable incident, cutting alert fatigue and letting the on-call engineer see the forest, not the trees.

4.2 4.2 Root-Cause Graph Engine — GNN-Based Causal Inference

The platform maintains a live **service-dependency graph** (nodes = services/pods/databases, edges = call relationships and resource dependencies). A **Graph Neural Network (GNN)** propagates anomaly scores across this graph and performs causal ranking to identify which node is the *most likely origin* of the anomaly propagation pattern — as opposed to simply flagging every node that shows abnormal metrics.

4.3 4.3 Safe, Closed-Loop Remediation — Policy-Gated Execution

Once a root cause and a candidate fix are identified (e.g., “rollback checkout-service to the previous stable version”), the action is **not executed blindly**. It passes through a **Policy Engine** (Open Policy Agent) that evaluates blast-radius, service criticality, time-of-day, and historical success rate of that action type before either (a) auto-executing low-risk actions, or (b) routing high-risk actions to a human for one-click approval.

4.4 4.4 LLM-Powered “Situation Room” — Natural Language Explanations

A Large Language Model, grounded via **Retrieval-Augmented Generation (RAG)** over the incident graph, telemetry snapshots, and historical postmortems, converts the raw causal graph into a **plain-English narrative**: what broke, why, what the blast radius is, and what the recommended fix is — replacing 45 minutes of dashboard-hopping with a 10-second read.

4.5 4.5 Predictive Failure Scoring — Proactive Maintenance

Beyond reacting to anomalies already in progress, the platform continuously computes a **Failure Risk Score** per service using leading indicators (memory growth slope, error-rate trend, deployment recency, historical incident frequency), enabling teams to intervene *before* a threshold breach becomes a customer-facing outage.

#	Feature	Primary Technique	Business Value
1	AI Incident Clustering	Embedding similarity + DBSCAN/temporal clustering	Reduces alert volume by up to 95%
2	Root-Cause Graph Engine	Graph Neural Networks (GNN) + causal ranking	Cuts diagnosis time from ~40 min to seconds
3	Safe Closed-Loop Remediation	OPA policy gating + automated executors	Enables automation without unsafe blast radius
4	LLM Situation Room	LLM + RAG over incident context	Converts technical graphs into human narrative
5	Predictive Failure Scoring	Trend/slope-based risk models	Shifts posture from reactive to proactive

5 5. End-to-End Example Scenario (The User Journey)

This section narrates a full incident lifecycle through the eyes of **Sarah**, an on-call SRE, to make the platform’s value tangible to a reader who has never seen it operate.

Step 1 — The Storm Begins (00:00). At 2:14 AM, a slow memory leak in `checkout-service` (introduced by a deployment six hours earlier) finally exhausts available heap. Within 90 seconds, **512 individual alerts** fire across Prometheus (high memory), the service mesh (elevated p99 latency on 14 downstream callers), and the log pipeline (a burst of `OutOfMemoryError` log lines). In a legacy setup, Sarah’s phone would explode with 500+ PagerDuty notifications.

Step 2 — Clustering Kicks In (00:02). AIOps Sentinel’s **Incident Clustering** engine ingests the 512 raw alerts within the same 90-second window, computes embedding similarity and topology proximity, and collapses them into **one Situation**: *“SIT-4471: Elevated latency & memory pressure — checkout-service and 14 dependents.”* Sarah receives **one** page, not 512.

Step 3 — Root Cause Localization (00:03). The **Root-Cause Graph Engine** ingests the live dependency graph and propagates the anomaly scores. Even though 15 services show abnormal metrics, the GNN’s causal ranking identifies `checkout-service` — specifically pod group `checkout-service-7f9c`, deployed 6 hours prior — as the origin node with 94% confidence, because its anomaly signal *precedes* and *predicts* all 14 downstream anomalies in the graph.

Step 4 — Plain-English Explanation (00:04). Sarah opens the **Situation Room**. Instead of 15 dashboards, she reads a single LLM-generated paragraph: *“Checkout-service’s memory usage grew steadily after the 8:00 PM deployment and crossed the OOM threshold at 2:14 AM. This is consistent with a memory-leak pattern rather than a traffic spike, since request volume was flat. 14 downstream services are experiencing cascading latency because they depend on checkout-service’s synchronous API. Recommended action: rollback checkout-service to the previous stable release (v2.114), which does not exhibit this growth pattern.”*

Step 5 — Policy-Gated Remediation Proposal (00:05). The **Remediation Planner** drafts the rollback action. The **Safety Guardrail (OPA)** evaluates it: rollback is a pre-approved action type for this service, but because `checkout-service` is tagged **tier-0 (revenue-critical)**, the policy requires **human approval** rather than fully autonomous execution.

Step 6 — One-Click Approval (00:06). Sarah reviews the explanation and the diff between the current and target version, and clicks **“Approve & Execute.”** The Remediation Executor issues a Kubernetes rollout-undo command against the specific deployment.

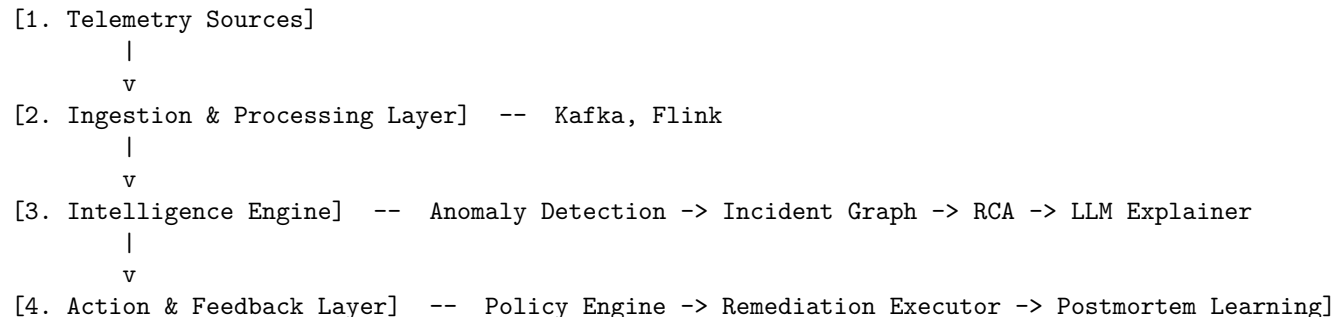
Step 7 — Verification & Closure (00:09). Within 3 minutes, memory usage returns to baseline and the 14 downstream latency alerts self-clear. The platform automatically marks the Situation as **Resolved** and generates a structured timeline (detection → clustering → root cause → approval → remediation → recovery) that seeds the postmortem document.

Outcome: A failure that would traditionally have taken **35–50 minutes** of manual triage across 15 dashboards and multiple engineers is resolved in **under 10 minutes** by a single engineer with full confidence in *why* the action was taken.

6. System Design & Reference Architecture

6.1 Architecture Pipeline Overview

AIOps Sentinel is organized into four sequential layers, each with a clear contract with its neighbors:



6.2 Layer-by-Layer Breakdown

6.2.1 Layer 1: Telemetry Sources

Raw signal producers already present in any Kubernetes-based stack:

- **Metrics:** Prometheus exporters (node, pod, application-level custom metrics)
- **Logs:** Loki / Fluentd log streams from every container
- **Traces:** OpenTelemetry-instrumented distributed traces (Jaeger backend)
- **Events:** Kubernetes API server events (pod evictions, OOMKills, scheduling failures)
- **Deployment metadata:** CI/CD pipeline webhooks (what changed and when)

6.2.2 Layer 2: Ingestion & Processing (Kafka / Flink)

All raw telemetry is published onto **Kafka topics** (one topic family per signal type: `metrics.raw`, `logs.raw`, `traces.raw`, `events.k8s`). **Apache Flink** jobs consume these topics to perform:

- Windowed aggregation (e.g., 30-second rolling averages per service)
- Schema normalization (mapping heterogeneous exporter formats into one canonical event schema)
- Early-stage feature extraction (rate-of-change, z-scores) that will feed the Anomaly Engine

This layer exists specifically to **decouple ingestion throughput from downstream ML inference latency** — Kafka provides durable buffering so that a burst of 10,000 events/sec during an actual incident does not overwhelm the intelligence layer.

6.2.3 Layer 3: Intelligence Engine

This is the analytical core, composed of four sequential sub-stages (each detailed further in Section 8):

1. **Anomaly Detection** — flags individual signal deviations (per-service, per-metric)
2. **Incident Graph Builder** — clusters correlated anomalies and maintains the live dependency graph
3. **Root Cause Analyzer** — performs causal ranking over the graph to identify origin nodes
4. **LLM Explainer** — converts the structured causal output into natural-language narrative, grounded via RAG over historical incidents and runbooks

6.2.4 Layer 4: Action & Feedback

1. **Policy Engine (OPA)** — evaluates every proposed remediation against organizational safety rules
2. **Remediation Executor** — carries out approved actions against the Kubernetes API (rollback, restart, scale, circuit-break)

3. **Postmortem Learning Loop** — every resolved incident’s full timeline, root cause, and remediation outcome is written back into the historical dataset, which **improves future clustering, RCA confidence scoring, and policy trust weights** — closing the loop from “one-off fix” to “continuously learning system”

6.3 6.3 Inter-Layer Interaction Summary

From	To	Interaction Contract
Telemetry Sources	Ingestion	Push raw events via exporters/agents onto Kafka topics
Ingestion	Intelligence Engine	Flink emits normalized, windowed feature vectors consumed by anomaly models
Anomaly Detection	Incident Graph	Anomaly scores per node are attached to the live topology graph
Incident Graph	RCA	GNN propagates scores across edges to rank candidate root-cause nodes
RCA	LLM Explainer	Structured causal output (root node, confidence, affected nodes) becomes the RAG context
LLM Explainer	Situation Room UI	Narrative + recommended action rendered to the SRE
Remediation Planner	Policy Engine	Every proposed action is intercepted and evaluated before execution
Remediation Executor	Postmortem Learning	Outcome (success/failure/rollback) is logged and fed back into training data

7 7. Deep Research: Datasets, Metrics & Research Gap

7.1 7.1 Candidate Datasets

Dataset	Modality	Relevance to This Project
OpenTelemetry AIOps Benchmark	Traces, metrics, logs (multi-modal, labeled anomalies)	Primary benchmark for training and evaluating the multi-modal anomaly detector
Microsoft Services Dataset (Public Incident Traces)	Service dependency graphs, incident timelines	Used to validate the Root-Cause Graph Engine against real large-scale service topologies
LO2 Microservice API Anomaly Dataset	Logs + API call traces from a realistic microservice testbed	Used for fine-tuning the anomaly detector on log-sequence patterns and for generating synthetic alert storms for the clustering engine

Each dataset contributes a distinct modality; combining them supports the project’s **multi-modal fusion** research angle (Section 7.3), since a model trained on only one modality (e.g., metrics alone) cannot distinguish a genuine memory leak from a legitimate traffic-driven scaling event without corroborating log and trace evidence.

7.2 7.2 Target Evaluation Metrics

Metric	Target	Why It Matters
Anomaly Detection F1-Score	$\geq$ 0.90	Balances false positives (alert fatigue) against false negatives (missed incidents)
Anomaly Detection AUC-ROC	$\geq$ 0.95	Measures overall discriminative power of the anomaly model across thresholds
MTTR Reduction	40–50% vs. manual baseline	The primary business-value metric — demonstrates real operational impact
False Remediation Rate	$<$ 5%	Critical safety metric — a remediation action taken on the wrong root cause must be rare
Incident Clustering Compression Ratio	Target $>$ 90% alert volume reduction	Demonstrates the alert-storm-reduction claim quantitatively
Root Cause Top-1 Accuracy	$\geq$ 85%	Measures how often the GNN’s top-ranked node is the true injected root cause in labeled test scenarios

7.3 7.3 Research Gap & IEEE Publication Angle

Existing AIOps literature tends to address these three sub-problems **in isolation**:

1. Multi-modal anomaly detection (fusing metrics + logs + traces) is studied, but typically stops at *detection*, not *causal explanation*.
2. Explainable root-cause analysis using graph-based causal inference is studied, but rarely connects its output to *natural-language explanation* usable by a non-expert on-call engineer.

3. Autonomous remediation is studied largely in narrow, single-action contexts (e.g., autoscaling), without a generalized, policy-gated safety framework spanning arbitrary remediation types.

AIOps Sentinel’s research contribution is the **integration** of all three: a system that performs multi-modal fusion for detection, explainable GNN-based causal inference for root cause, and a generalized OPA-based policy framework for safe autonomous remediation — evaluated end-to-end on MTTR and false-remediation-rate rather than on detection accuracy alone.

Suggested IEEE paper title:

“AIOps Sentinel: A Multi-Modal, Explainable, and Policy-Governed Framework for Closed-Loop Cloud Self-Healing”

8 8. AI/ML Core Functional Modules

Module	Function	AI/ML Technique
Telemetry Collector	Ingests raw metrics, logs, and traces from the cluster	Prometheus (metrics), Loki (logs), Jaeger (traces) — instrumentation layer, not ML itself
Anomaly Engine	Detects statistically abnormal behavior per service/metric	DAGMM (Deep Autoencoding Gaussian Mixture Model) for multi-dimensional metric anomalies; Isolation Forest as a fast, interpretable baseline; Transformer Autoencoder for sequential log/trace anomaly detection
Incident Graph Builder	Constructs and maintains the live service-dependency graph, and clusters correlated anomalies into one Situation	Graph Neural Networks (GNNs) for representation learning over the topology; embedding-based clustering for alert deduplication
Root Cause Analyzer	Ranks candidate root-cause nodes among all anomalous nodes	Causal inference on the GNN (score propagation + temporal precedence weighting) to distinguish origin nodes from downstream victims
Remediation Planner	Proposes a concrete corrective action given the identified root cause	LLM-backed planning — the LLM matches the diagnosed failure pattern against a curated library of remediation playbooks and drafts the specific action (e.g., which deployment to roll back to which version)
Safety Guardrail	Gates every proposed action against organizational risk policy	Open Policy Agent (OPA) — declarative Rego policies evaluate blast radius, service tier, action risk class, and time-of-day before allowing auto-execution vs. requiring human approval
LLM Explainer	Converts structured causal + telemetry data into a natural-language incident narrative	LLM + Retrieval-Augmented Generation (RAG) — retrieves relevant historical incidents, runbook excerpts, and current telemetry snapshots as grounding context before generation, minimizing hallucination

9. Tech Stack & Database Selection

9.1 Recommended Technology Stack

Layer	Technology Choices	Rationale
Frontend	React, Stitch AI (for rapid UI scaffolding), Tailwind CSS	Component-driven UI development with fast iteration on the dark “Mission Control” design system
Backend	FastAPI (Python, for ML-serving endpoints), Node.js (for real-time WebSocket/event-driven services)	FastAPI’s native async support pairs naturally with PyTorch inference; Node.js excels at low-latency event streaming to the frontend
AI/ML	PyTorch Geometric (GNN implementation), Scikit-learn (Isolation Forest baseline), LangChain (RAG orchestration and LLM tool-calling)	Purpose-built libraries for each specific ML technique named in Section 8
Streaming	Apache Kafka, Apache Flink	Industry-standard durable event streaming and stateful stream processing at scale
Orchestration	Kubernetes	Both the platform’s own deployment target <i>and</i> the remediation target being managed — enabling native integration with the Kubernetes API for executor actions

9.2 Database Selection: Supabase vs. Firebase

Criterion	Supabase (PostgreSQL)	Firebase (NoSQL / Firestore)
Data model fit for RCA graphs	Strong — relational tables + pgvector extension can represent both structured incident records and graph edge lists efficiently via SQL joins	Weak — document-based NoSQL model does not naturally express relational graph traversal queries
Native vector embeddings for LLM/RAG	Built-in via pgvector extension — embeddings for incidents, alerts, and runbooks can be stored and queried (cosine similarity) in the same database as the relational data	Requires a separate vector database (e.g., Pinecone) bolted on externally, adding architectural complexity
Complex relational queries (e.g., join alerts → incidents → remediation history)	Native SQL joins, aggregations, window functions	Requires denormalization or multiple round-trip reads; joins are not first-class
Schema enforcement	Strong typing and constraints (foreign keys, enums) — critical for an auditable safety-relevant system	Schema-less by default, increasing risk of inconsistent incident records

Criterion	Supabase (PostgreSQL)	Firebase (NoSQL / Firestore)
Open-source / self-hostable	Yes — avoids vendor lock-in, important for an enterprise B2B platform	No — proprietary Google Cloud service

Verdict: Supabase (PostgreSQL) is the clear winner for AIOps Sentinel. The platform’s two most data-intensive needs — (1) representing and querying the **relational, graph-like structure** of incidents/alerts/services, and (2) storing **vector embeddings** for the LLM’s RAG retrieval step — are both natively satisfied by PostgreSQL plus the **pgvector** extension inside a single database, avoiding the operational overhead of stitching together a document store and a separate vector database.

9.3 Database Schema (SQL)

```
-- Core incident record (one row per clustered "Situation")
CREATE TABLE incidents (
  incident_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title            TEXT NOT NULL,
  status           TEXT NOT NULL CHECK (status IN ('open','investigating','remediating','resolved')),
  root_cause_service TEXT,
  root_cause_confidence NUMERIC(5,2),
  severity         TEXT CHECK (severity IN ('sev1','sev2','sev3','sev4')),
  narrative_summary TEXT,                                -- LLM-generated explanation
  narrative_embedding VECTOR(1536),                    -- pgvector: for RAG similarity search
  detected_at      TIMESTAMPTZ NOT NULL,
  resolved_at      TIMESTAMPTZ,
  created_at       TIMESTAMPTZ DEFAULT now()
);

-- Raw alerts, later clustered into incidents
CREATE TABLE alerts (
  alert_id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  incident_id      UUID REFERENCES incidents(incident_id),
  source_system    TEXT NOT NULL,                      -- e.g. 'prometheus', 'jaeger', 'loki'
  service_name     TEXT NOT NULL,
  metric_name      TEXT,
  alert_payload    JSONB NOT NULL,
  fired_at         TIMESTAMPTZ NOT NULL,
  embedding        VECTOR(1536)                        -- for similarity-based clustering
);

-- Library of known remediation actions ("playbooks")
CREATE TABLE remediation_playbooks (
  playbook_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  failure_pattern  TEXT NOT NULL,                      -- e.g. 'memory-leak', 'connection-pool-exhaustion'
  action_type      TEXT NOT NULL CHECK (action_type IN ('rollback','restart','scale_out','circuit_br'),
  risk_class       TEXT CHECK (risk_class IN ('low','medium','high')),
  success_rate     NUMERIC(5,2) DEFAULT 0,
  playbook_body    TEXT NOT NULL,                      -- human-readable runbook text used as RAG context
  created_at       TIMESTAMPTZ DEFAULT now()
);

-- Time-series telemetry snapshots referenced by incidents (for traceability/audit)
```

```
CREATE TABLE telemetry_metrics (  
  metric_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  incident_id    UUID REFERENCES incidents(incident_id),  
  service_name   TEXT NOT NULL,  
  metric_name    TEXT NOT NULL,  
  metric_value   DOUBLE PRECISION NOT NULL,  
  recorded_at    TIMESTAMPTZ NOT NULL  
);
```

10 10. UI/UX Specifications (For Stitch AI / Lovable)

10.1 10.1 Design System

Element	Specification
Theme	Dark-mode “Mission Control” — deep blue/near-black background (#0A0E17)
Accent Colors	Cyan (#22D3EE, healthy/info), Amber (#F59E0B, warning/investigating), Red (#EF4444, critical/sev1)
Typography	Inter for UI text and body copy; JetBrains Mono for all logs, metrics, and code/command output
Layout Skeleton	Fixed left sidebar (primary navigation), top bar (global search, active-incident count, user profile), and a fluid main content area

10.2 10.2 Pages & Navigation (7 Pages)

#	Page	Purpose
1	Login / Auth	Secure entry point; SSO-ready
2	Mission Control Dashboard	At-a-glance system health: active situations, failure-risk heatmap, recent deployments
3	Situation Room (Incident Detail)	Deep-dive view of one clustered incident: timeline, LLM narrative, affected services
4	Alert Clustering View	Shows the raw alert stream and how it was compressed into situations
5	Root Cause Graph Explorer	Interactive service-dependency graph with anomaly heat overlay
6	Remediation Playbooks	Library of known fixes, success rates, and risk classifications
7	Settings & Policies	Configure OPA policy rules, service tiers, and notification routing

10.3 10.3 Page-Level Interactions

Page	Key Interactive Elements
Mission Control Dashboard	“ Simulate Incident ” button (for demos/training), clickable situation cards that deep-link into the Situation Room
Situation Room	“ Explain ” button (regenerates/expands the LLM narrative), “ Approve ” / “ Deny ” buttons on the proposed remediation card, live timeline scrubber
Alert Clustering View	Toggle between “raw alert stream” and “clustered view”; click-to-expand cluster membership
Root Cause Graph Explorer	Click a node to see its metrics; drag-to-pan/zoom on the graph canvas; anomaly-intensity color overlay

Page	Key Interactive Elements
Remediation Playbooks	“Edit Playbook” , “Test in Dry-Run Mode” buttons
Settings & Policies	Rego policy editor with syntax highlighting; “Save & Validate Policy” button

11 11. Risks, Constraints & Mitigation

Risk / Constraint	Description	Mitigation Strategy
Bad automation worsening outages	An incorrect root-cause diagnosis could trigger a remediation action that makes the outage worse (e.g., rolling back a <i>good</i> deployment)	Enforce the Safety Guardrail’s confidence thresholds; require human approval for any action on tier-0/critical services regardless of confidence score
Human approval boundary	Full autonomy is not acceptable for high-blast-radius actions in a production financial or revenue-critical system	OPA policies explicitly encode which action/service/risk combinations require a human-in-the-loop approval versus full auto-execution
Proving reliability before full autonomy	Stakeholders will not trust an autonomous system without a track record	Launch in “ shadow mode ” first (recommend actions but never execute), measure recommendation accuracy against real incidents for a defined trial period, then graduate low-risk action types to auto-execution only after success-rate thresholds are met
False positives causing alert fatigue	An overly sensitive anomaly detector generates noise that erodes trust	Continuously tune detection thresholds using the F1/AUC-ROC targets in Section 7.2; incorporate engineer feedback (thumbs up/down on each Situation) as a retraining signal
LLM hallucination in explanations	The LLM could state an incorrect root cause or fabricate a plausible-sounding but false narrative	Ground every LLM generation in RAG-retrieved structured evidence (the actual causal graph output and telemetry snapshots), never let the LLM free-generate the root cause itself
Data privacy / multi-tenant isolation	As a B2B platform, telemetry from different customer organizations must never cross-contaminate	Enforce row-level security in Supabase/PostgreSQL per tenant; isolate Kafka topics per customer namespace

12 12. Conclusion & PPO Pitch Strategy

12.1 12.1 Why This Project Is Industry-Ready

AIOps Sentinel is not a toy demo — it mirrors the exact architecture patterns (event streaming, graph-based ML, LLM grounding via RAG, policy-as-code safety) used in production observability platforms at major cloud and SaaS companies. It demonstrates competency across **five distinct engineering disciplines** simultaneously: distributed systems (Kafka/Flink), machine learning (GNNs, autoencoders), applied LLM engineering (RAG, prompt grounding), platform/DevOps engineering (Kubernetes, OPA), and full-stack product engineering (React + FastAPI + PostgreSQL).

12.2 12.2 Pitching This to Recruiters for a PPO

When presenting this project in a placement interview, structure the pitch around three pillars:

1. **Real-World Impact, Quantified.** Lead with the metrics from Section 7.2 — a 40–50% MTTR reduction and a >90% alert-volume compression are business outcomes a recruiter immediately understands, not just academic accuracy numbers.
2. **Technical Depth Across the Stack.** Be ready to go deep on *any* layer if asked: explain why a GNN (rather than a simple threshold rule) is necessary for root-cause ranking, why `pgvector` was chosen over a separate vector database, and why OPA (rather than hard-coded if/else logic) is the right abstraction for safety policy.
3. **Production-Readiness Mindset.** Emphasize the **shadow-mode rollout strategy** from Section 11 — this signals to recruiters that you think like an engineer who has to earn trust in a real production environment, not just someone who shipped a model with a high F1-score. This maturity — knowing *when not* to fully automate — is precisely what distinguishes a strong systems engineer from a student project.

Closing line for the pitch: *“AIOps Sentinel doesn’t just detect that something is wrong — it explains why, in plain English, and safely fixes it, cutting incident resolution time in half while keeping a human in control of every high-risk decision.”*